



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Название: Работа со стеком

Дисциплина: Типы и структуры данных

Вариант: 18

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Силантьева А.В.

Оглавление

Условие задачи	3
Описание ТЗ.....	3
1. Описание входных данных	3
2. Описание результатов работы программы	4
3. Способ обращения к программе	4
4. Задача программы	4
5. Описание возможных аварийных ситуаций и ошибок пользователя....	4
Описание внутренних структур данных	5
Описание алгоритма работы	6
Тестовые случаи	6
Сравнение эффективности двух типов стеков при разных строках- выражениях	7
Выводы	7
Ответы на контрольные вопросы	8

Условие задачи

Создать программу работы со стеком, выполняющую операции добавления, удаления элементов и вывод текущего состояния стека.

Реализовать стек: а) массивом; б) списком. Все стандартные операции со стеком должны быть оформлены подпрограммами. При реализации стека списком в вывод текущего состояния стека добавить просмотр адресов элементов стека и создать свой список или массив свободных областей (адресов освобождаемых элементов) с выводом его на экран.

Операция со стеком по варианту: перевести выражение в постфиксную форму.

Описание ТЗ

1. Описание входных данных

На вход программе из файлов поступает строка-выражение вида:

$(A+B)*C$

Ограничения:

1. Строка-выражение для ввода всегда корректна
2. Для обозначения операций в строке-выражении могут использоваться только следующие символы:
 - «+» - сложить
 - «-» - вычесть
 - «*» - умножить
 - «/» - разделить
3. Допустимо использовать круглые скобки для обозначения приоритетности операций
4. Для обозначения операндов в строке-выражении могут использоваться только заглавные и строчные буквы латинского алфавита

5. Любой код, не лежащий в диапазоне от 0 до 5 включительно, считается ошибочным

2. Описание результатов работы программы

В результате работы программы создаются 2 переменные типа «стек-массив» и «стек-список». После этого выводится меню программы, содержащее в себе следующие пункты:

- 0) Выход
- 1) Вывести стек
- 2) Добавить элемент в стек
- 3) Удалить элемент из стека
- 4) Выполнить преобразование выражения с постфиксную форму записи при помощи стека
- 5) Сравнить время выполнения операций и объем памяти при использовании двух типов стеков

3. Способ обращения к программе

Обращение к программе происходит из командной строки путем запуска исполняемого файла «*app.exe*».

4. Задача программы

Задача данной программы заключала в себе несколько составных частей:

- Преобразование строки-выражения в постфиксную форму при помощи двух типов стеков
- Измерение времени и памяти, необходимых для преобразования строки-выражения в постфиксную форму

5. Описание возможных аварийных ситуаций и ошибок пользователя

- Ошибка при пустом стеке:
 - Код возврата: 1
 - Сообщение: «Стек пуст. Сначала добавьте элементы в него.»

- Некорректный код меню:
 - Код возврата: 2
 - Сообщение: «Некорректный код. Попробуйте снова.»
- Ошибка выделения памяти:
 - Код возврата: 3
 - Сообщение: «Не удалось выделить память. Попробуйте снова.»
- Ошибка при переполнении стека:
 - Код возврата: 4
 - Сообщение: «Стек полностью занят. Удалите элементы и попробуйте снова.»
- Введена слишком длинная строка-выражение:
 - Код возврата: 5
 - Сообщение: «Выражение слишком большое. Попробуйте снова.»
- Введена некорректная строка-выражение:
 - Код возврата: 6
 - Сообщение: «Выражение для преобразования некорректное. Попробуйте снова.»

Описание внутренних структур данных

Тип данных, описывающий структуру для работы со стеком в виде массива:

```
typedef struct
{
    char content[STACK_SIZE]; // Элементы стека
    int len;                  // Кол-во элементов в стеке
} arr_stack_t;
```

Типы данных, описывающие структуры для работы со стеком в виде списка:

```
typedef struct node node_t;

struct node
{
    char item; // Элемент стека
    node_t *next; // Указатель на следующий узел
};
```

```
typedef struct
{
    node_t *top; // Вершина стека-списка
    int len; // Кол-во элементов в стеке
} list_stack_t;
```

Тип данных, описывающий структуру для хранения адресов освобожденных узлов стека-списка:

```
typedef struct
{
    node_t *p_nodes[STACK_SIZE]; // Адреса
    int len; // Кол-во адресов
} p_node_t;
```

Тип данных, описывающий структуру для хранения результатов замерного эксперимента:

```
typedef struct measure
{
    long long unsigned time; // Время
    int mem; // Память
    char expr[EXPR_LEN + 1]; // Выражение
} measurement_table;
```

Описание алгоритма работы

1. Пользователю выводится меню программы с возможностью выбрать желаемый
2. Пользователь вводит желаемый пункт меню
3. Пока пользователь не введет «0» (пункт выхода из программы), ему будет предложено вводить номера меню и выполнять желаемые действия

Тестовые случаи

Позитивные тесты:

1. Обычная строка-выражение
2. Строка-выражение из одного элемента
3. Длина строки-выражения равна максимальной длине
4. В строке-выражении незначащие скобки

Негативные тесты:

1. Строка-выражение слишком большая
2. Строка-выражение содержит незакрытые круглые скобки

Сравнение эффективности двух типов стеков при разных строках-выражениях

Таблица замеров для различных строк-выражений:

Выражение для преобразования	Время 1000 выполне ний (массив, мкс)	Время 1000 выполне ний (список, мкс)	Памя ть (масс ив, байт)	Памя ть (спис ок, байт)
$A+B$	17	34	44	32
$(A+B)*C - (D-E)*(F+G)$	80	242	44	80
$((((A+B)*C) - (D-E))*(F+G))$	95	300	44	208
$(((((A+B)*C) - (D-E))*((F+G)))/H) - L$	166	398	44	304
$A+A+A+A+A+A+A+A+A+A+A+A+A+A+A+A$	195	631	44	400

Выводы

В ходе работы были изучены способы и алгоритмы работы со стеками, а также проведены замерные эксперименты, по результатам которых можно сделать вывод, что преобразование строки-выражения в постфиксную форму выполняется быстрее примерно в 2-3 раза при помощи стека-массива, однако при небольших введенных выражениях (например, « $A+B$ ») стеку в виде списка необходимо меньше памяти, что позволяет программисту выбрать такой вариант обработки, если существует необходимость сэкономить на тратах памяти.

Ответы на контрольные вопросы

- 1) Стек – это последовательный список с переменной длиной, в котором включение и исключение элементов происходит только с одной стороны – с его вершины. Стек функционирует по принципу: LIFO - последним пришел – первым ушел
- 2) При хранении стека в виде списка, память выделяется в куче, а при хранении в виде массива – либо в куче, либо в стеке (зависит от типа используемого массива). Для каждого узла списка выделяется в среднем на 4-8 байт больше, так как в нем хранится еще указатель на следующий элемент списка
- 3) Для стека-массива память освобождается на этапе завершения программы, так как для удаления элемента такого стека достаточно уменьшить переменную длины на 1. Для стека-списка память освобождается в тот момент, когда удаляется элемент и указатель на вершину меняется на значение следующего узла
- 4) Элементы стека удаляются, так как просмотр возможен только в случае удаления просмотренного элемента
- 5) Стек эффективнее реализовать с помощью массива, так как он выигрывает в количестве занимаемой памяти (в обычном случае) и во времени обработки стека.